MODUL 10 · BAB IV · DYNAMIC PROGRAMMING

Konsep Dynamic Programming

Menyelesaikan masalah optimasi dengan menyimpan solusi submasalah agar tidak dihitung berulang kali.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan jenis masalah yang cocok diselesaikan dengan Dynamic Programming (masalah optimasi).
- Menjelaskan dua ciri utama: *optimal substructure* dan *overlapping subproblems*.
- Menerapkan empat langkah Dynamic Programming pada contoh sederhana.
- Membedakan pendekatan *top-down* (memoization) dan *bottom-up* (tabel).
- Membedakan Dynamic Programming dari Divide and Conquer dan menghitung running time-nya.

PRASYARAT

- Modul 02: rekursi.
- Modul 08: paradigma Divide and Conquer.
- Modul 05–06: notasi asimtotik dan pertumbuhan eksponensial.

BAB / MATERI ASAL

Materi 8 DYNAMIC PROGRAMMING Perkalian Rantai Matriks (bagian pengantar Dynamic Programming)

POSISI DALAM URUTAN BELAJAR

Konsep umum Bab IV. Diterapkan secara khusus pada Perkalian Rantai Matriks (Modul 11) dan LCS (Modul 12).

DAFTAR ISI

1. Ketika Rekursi Mengerjakan Hal yang Sama Berulang-ulang 1
2. Masalah Optimasi 2
3. Dua Ciri Masalah Dynamic Programming 2
4. Empat Langkah Dynamic Programming 3
5. Top-down dan Bottom-up 4
6. Running Time dan Perbandingan dengan Paradigma Lain 5
7. Kesimpulan 6
- Kuis Pemahaman 8
- Kunci Jawaban dan Pembahasan 9
- Referensi 9

PENDAHULUAN

1. Ketika Rekursi Mengerjakan Hal yang Sama Berulang-ulang

Divide and Conquer (Modul 08) memecah masalah menjadi submasalah yang **terpisah**: bagian kiri dan bagian kanan Merge Sort tidak pernah mengerjakan hal yang sama. Namun ada banyak masalah yang ketika dipecah menghasilkan submasalah yang **sama** berulang kali. Jika diselesaikan dengan rekursi biasa, submasalah yang sama dihitung lagi dan lagi, dan running time bisa membengkak secara eksponensial.

Dynamic Programming mengatasi masalah ini dengan prinsip yang sangat sederhana: **selesaikan setiap submasalah sekali saja, simpan jawabannya di tabel, dan gunakan kembali bila dibutuhkan**. Istilah ini diperkenalkan oleh Richard Bellman pada 1950-an; kata *programming* di sini berarti "perencanaan dengan tabel", bukan menulis kode program.

Modul ini membahas konsep umum Dynamic Programming: masalah apa yang cocok, ciri-ciri yang harus dimiliki masalah tersebut, empat langkah penyelesaiannya, dan cara menghitung running time-nya. Dua penerapan lengkap dari bahan kuliah, yaitu **Perkalian Rantai Matriks** dan **Longest Common Subsequence**, dibahas di Modul 11 dan 12.

PEMBAHASAN INTI

2. Masalah Optimasi

Dynamic Programming umumnya dipakai untuk **masalah optimasi**: masalah yang memiliki **banyak solusi yang sah**, setiap solusi memiliki **nilai**, dan kita ingin mencari solusi dengan nilai **optimal** (maksimum atau minimum). Contoh dari bahan kuliah:

- **Perkalian rantai matriks**: banyak cara menempatkan tanda kurung; nilainya adalah banyaknya operasi perkalian skalar; dicari yang **minimum**.
- **Longest Common Subsequence**: banyak subbarisan yang dimiliki bersama oleh dua barisan; nilainya adalah panjang subbarisan; dicari yang **maksimum**.

Perhatikan bahwa yang dicari adalah *sebuah* solusi optimal. Bisa saja ada beberapa solusi berbeda dengan nilai optimal yang sama, seperti LCS di Modul 12 yang memiliki lebih dari satu jawaban dengan panjang 4.

DEFINISI

Dynamic Programming menyelesaikan masalah dengan **menggabungkan solusi-solusi submasalah**. Solusi setiap submasalah **disimpan dalam tabel**, sehingga ketika menghitung solusi submasalah yang lebih besar, solusi submasalah yang lebih kecil cukup diambil dari tabel tanpa dihitung ulang.

PEMBAHASAN INTI

3. Dua Ciri Masalah Dynamic Programming

Cormen dkk. (2009, Bab 15.3) menyebutkan dua ciri yang harus dimiliki suatu masalah agar Dynamic Programming dapat diterapkan.

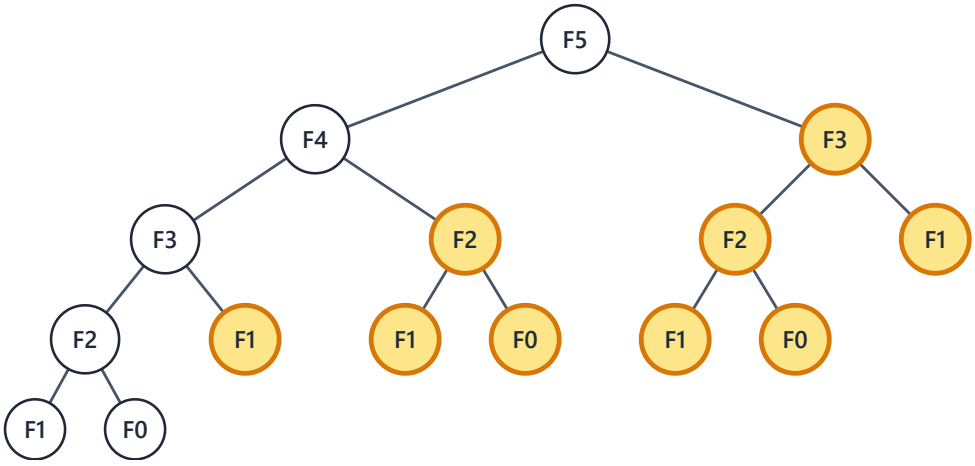
3.1 Optimal substructure (substruktur optimal)

Solusi optimal masalah tersusun dari **solusi optimal submasalahnya**. Contoh: jika cara terbaik mengalikan $A_1 A_2 \cdots A_6$ adalah memecahnya menjadi $(A_1 A_2 A_3)(A_4 A_5 A_6)$, maka cara mengalikan $A_1 A_2 A_3$ di dalamnya juga harus cara yang terbaik. Seandainya ada cara yang lebih murah untuk $A_1 A_2 A_3$, kita bisa menggantinya dan memperoleh solusi keseluruhan yang lebih murah, sehingga solusi semula ternyata tidak optimal. Argumen seperti ini disebut **argumen potong-dan-tempel** (*cut-and-paste*).

3.2 Overlapping subproblems (submasalah tumpang tindih)

Rekursi alami untuk masalah tersebut menyelesaikan **submasalah yang sama berulang kali**. Inilah pembeda utama dengan Divide and Conquer, yang submasalahnya selalu baru.

Contoh paling mudah untuk melihat tumpang tindih adalah bilangan Fibonacci: $F_0 = 0$, $F_1 = 1$, dan $F_n = F_{n-1} + F_{n-2}$. (Fibonacci bukan masalah optimasi, tetapi sangat jelas memperlihatkan pemborosan perhitungan ulang.)



Gambar 1. Pohon pemanggilan rekursif untuk F_5. Simpul berwarna kuning adalah perhitungan ulang dari submasalah yang sudah pernah dihitung.

Untuk menghitung F_5, submasalah F_3 dihitung 2 kali dan F_2 dihitung 3 kali. Banyaknya pemanggilan rekursif untuk F_n adalah $2F_{n+1} - 1$, yang tumbuh secara eksponensial: 15 pemanggilan untuk F_5, tetapi **2.692.537** pemanggilan untuk F_{30}. Padahal hanya ada 31 submasalah berbeda (F_0 sampai F_{30}).

PEMBAHASAN INTI

4. Empat Langkah Dynamic Programming

Bahan kuliah merumuskan penyelesaian masalah optimasi dengan Dynamic Programming dalam empat langkah berurutan:

1. **Karakterisasi struktur solusi optimal.** Tentukan bagaimana solusi optimal tersusun dari solusi optimal submasalah (optimal substructure).
2. **Definisikan nilai solusi optimal secara rekursif.** Tuliskan rumus rekursif yang menyatakan nilai optimal masalah dalam bentuk nilai optimal submasalah.
3. **Hitung nilai solusi optimal**, biasanya dengan cara **bottom-up**: isi tabel mulai dari submasalah terkecil.
4. **Susun solusi optimal** dari informasi yang dicatat selama perhitungan.

Langkah 1–3 menghasilkan **nilai** optimal (misalnya "minimal 15.125 operasi perkalian"). Langkah 4 hanya diperlukan bila kita juga ingin tahu **solusinya** (misalnya "urutan perkaliannya $(A_1(A_2A_3))((A_4A_5)A_6)$"). Untuk itu, selama langkah 3 biasanya dicatat pilihan terbaik di tabel kedua, seperti tabel s pada Modul 11 dan tabel b pada Modul 12.

4.1 Contoh ringkas: pemotongan batang

Untuk melihat keempat langkah dalam satu contoh kecil, kita pakai masalah pemotongan batang dari Cormen dkk. (2009, Bab 15.1). Contoh ini **tambahan**, bukan dari bahan kuliah. Sebuah batang sepanjang n dapat dipotong-potong menjadi bagian dengan panjang bilangan bulat. Bagian sepanjang i dapat dijual seharga p_i:

Panjang i	1	2	3	4	5	6	7	8
Harga p_i	1	5	8	9	10	17	17	20

Tabel 1. Harga jual potongan batang (Cormen dkk., 2009).

Kita ingin pendapatan maksimum r_n.

1. **Struktur:** pada solusi optimal, ada potongan pertama sepanjang i. Sisanya, sepanjang $n - i$, juga harus dipotong secara optimal.
2. **Rumus rekursif:** $r_0 = 0$ dan $r_n = \max_{1 \le i \le n} (p_i + r_{n-i})$.
3. **Bottom-up:** hitung $r_1, r_2, \ldots$ berurutan; setiap r_j hanya memakai nilai yang sudah ada di tabel.
4. **Susun solusi:** catat s_j = panjang potongan pertama terbaik untuk r_j.

j	1	2	3	4	5	6	7	8
r_j	1	5	8	10	13	17	18	22
s_j	1	2	3	2	2	6	1	2

Tabel 2. Tabel hasil bottom-up pemotongan batang.

Contoh r_4: pilihannya $p_1 + r_3 = 9$, $p_2 + r_2 = 10$, $p_3 + r_1 = 9$, $p_4 + r_0 = 9$. Maksimumnya 10 dengan $i = 2$, yaitu dua potongan sepanjang 2. Untuk $n = 8$: $s_8 = 2$, sisanya 6 dengan $s_6 = 6$, sehingga potongannya $2 + 6$ dengan pendapatan $5 + 17 = 22$.

PEMBAHASAN INTI

5. Top-down dan Bottom-up

Ada dua cara mengimplementasikan Dynamic Programming:

Aspek	Top-down dengan memoization	Bottom-up
Cara kerja	Tulis rekursi seperti biasa, tetapi sebelum menghitung, periksa apakah jawaban submasalah sudah ada di tabel (*memo*). Jika belum, hitung lalu simpan.	Urutkan submasalah dari yang terkecil, lalu isi tabel satu per satu dengan perulangan.
Submasalah yang dihitung	Hanya yang benar-benar dibutuhkan	Semua submasalah
Kelebihan	Dekat dengan rumus rekursif, mudah ditulis	Tanpa biaya rekursi, urutan perhitungan jelas
Contoh di bahan kuliah	-	MATRIX-CHAIN-ORDER, LCS-LENGTH

Tabel 3. Dua cara implementasi Dynamic Programming.

Keduanya memiliki running time asimtotik yang sama untuk sebagian besar masalah. Bahan kuliah memakai cara bottom-up, sesuai langkah 3.

Python: rekursi biasa vs memoization vs bottom-up

```python
from functools import lru_cache

panggilan = 0
def fib_naif(n):
    global panggilan
    panggilan += 1
    return n if n < 2 else fib_naif(n - 1) + fib_naif(n - 2)

@lru_cache(maxsize=None)            # top-down: hasil disimpan otomatis
def fib_memo(n):
    return n if n < 2 else fib_memo(n - 1) + fib_memo(n - 2)

def fib_bottom_up(n):
    F = [0, 1] + [0] * (n - 1)      # tabel F[0..n]
    for i in range(2, n + 1):
        F[i] = F[i - 1] + F[i - 2]
    return F[n]

print(fib_naif(30), "pemanggilan:", panggilan)
print(fib_memo(30), fib_bottom_up(30), fib_bottom_up(90))

p = [0, 1, 5, 8, 9, 10, 17, 17, 20]  # harga batang p[1..8]
r = [0] * 9
for j in range(1, 9):                # bottom-up pemotongan batang
    r[j] = max(p[i] + r[j - i] for i in range(1, j + 1))
print(r[1:])
```

Keluaran program

```
832040 pemanggilan: 2692537
832040 832040 2880067194370816120
[1, 5, 8, 10, 13, 17, 18, 22]
```

Versi bottom-up menghitung F_{90} dengan hanya 89 kali penjumlahan, sedangkan versi rekursif biasa akan membutuhkan sekitar 10^{19} pemanggilan untuk nilai yang sama.

ANALISIS DAN HUBUNGAN KONSEP

6. Running Time dan Perbandingan dengan Paradigma Lain

6.1 Menghitung running time Dynamic Programming

Karena setiap submasalah dihitung sekali, running time Dynamic Programming umumnya

$$\text{running time} = (\text{banyaknya submasalah}) \times (\text{banyaknya pilihan per submasalah}).$$

Masalah	Banyak submasalah	Pilihan per submasalah	Running time
Fibonacci	$n+1$	1 (penjumlahan tetap)	$\Theta(n)$
Pemotongan batang	$n+1$	sampai n	$\Theta(n^2)$
Perkalian rantai matriks (Modul 11)	$\Theta(n^2)$ pasangan (i, j)	sampai $n-1$ letak pemecah k	$\Theta(n^3)$
LCS (Modul 12)	$(m+1)(n+1)$ sel tabel	konstan (3 kemungkinan)	$\Theta(mn)$

Tabel 4. Running time beberapa algoritma Dynamic Programming.

6.2 Mengapa tidak brute force saja?

Mencoba semua solusi biasanya eksponensial. Bahan kuliah menunjukkan bahwa empat matriks dapat dikalikan dengan 5 cara penempatan kurung. Banyaknya cara untuk n matriks mengikuti barisan bilangan Catalan: 1, 1, 2, 5, 14, 42, 132, 429, 1.430, 4.862 untuk $n = 1, \ldots, 10$, dan tumbuh paling sedikit secara eksponensial, $\Omega(2^n)$ (Cormen dkk., 2009, Bab 15.2). Dynamic Programming memotong pertumbuhan itu menjadi polinomial karena submasalah yang sama, misalnya "cara terbaik mengalikan $A_2 \cdots A_4$", hanya dihitung sekali walaupun muncul di banyak susunan kurung.

6.3 Dibandingkan dengan Divide and Conquer dan Greedy

Paradigma	Submasalah	Cara memilih	Contoh
Divide and Conquer (Modul 08)	Saling lepas (tidak tumpang tindih)	Tidak ada pilihan; pembagian tetap	Merge Sort
Dynamic Programming	Tumpang tindih, solusinya disimpan	Mencoba semua pilihan, ambil yang terbaik berdasarkan solusi submasalah	Rantai matriks, LCS
Greedy (Modul 13)	Satu submasalah tersisa setelah memilih	Langsung mengambil pilihan yang terlihat terbaik saat itu	Activity selection, Huffman

Tabel 5. Tiga paradigma perancangan algoritma.

Bab berikutnya (Greedy) dimulai dengan pengamatan dari bahan kuliah: untuk sebagian masalah optimasi, mencoba semua pilihan seperti Dynamic Programming ternyata "berlebihan", karena pilihan yang terlihat terbaik saat itu sudah pasti benar.

KESIMPULAN

7. Kesimpulan

- Dynamic Programming menyelesaikan masalah optimasi dengan menggabungkan solusi submasalah dan menyimpannya di tabel agar tidak dihitung ulang.
- Syaratnya: masalah memiliki *optimal substructure* dan *overlapping subproblems*.
- Empat langkah: karakterisasi struktur solusi optimal, definisi nilai optimal secara rekursif, perhitungan bottom-up, dan penyusunan solusi optimal.
- Implementasi dapat top-down (memoization) atau bottom-up (tabel); bahan kuliah memakai bottom-up.
- Running time ≈ banyaknya submasalah × banyaknya pilihan per submasalah; sering mengubah brute force eksponensial menjadi polinomial.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Ciri yang membedakan masalah yang cocok untuk Dynamic Programming dari masalah yang cocok untuk Divide and Conquer biasa adalah ...

A. Masalahnya dapat dipecah menjadi dua bagian.
B. Submasalahnya tumpang tindih sehingga submasalah yang sama muncul berulang kali.
C. Masalahnya selalu berupa pengurutan.
D. Masalahnya tidak dapat diselesaikan secara rekursif.

2 ANALISIS

Jelaskan dengan kata-katamu sendiri arti *optimal substructure*, dan berikan alasan mengapa sifat ini diperlukan agar Dynamic Programming menghasilkan jawaban yang benar.

3 PERHITUNGAN

Berapa kali F_2 dihitung oleh rekursi biasa ketika menghitung F_6? Berapa total pemanggilan rekursifnya?

4 PERHITUNGAN

Dengan tabel harga pemotongan batang di modul ini, hitung r_5 secara manual dan tentukan cara pemotongannya.

5 BENAR/SALAH DENGAN ALASAN

"Langkah ke-4 Dynamic Programming (menyusun solusi optimal) wajib dilakukan pada setiap masalah." Benar atau salah?

6 PERHITUNGAN

Sebuah algoritma Dynamic Programming memiliki tabel berukuran $n \times n$, dan setiap sel dihitung dengan memeriksa paling banyak n sel lain. Berapa running time-nya? Berapa memori yang dibutuhkan?

7 PILIHAN GANDA

Apa perbedaan utama memoization (top-down) dan bottom-up?

A. Memoization tidak menyimpan hasil submasalah.
B. Bottom-up hanya menghitung submasalah yang dibutuhkan.
C. Memoization tetap memakai rekursi tetapi menyimpan hasil; bottom-up mengisi tabel dengan perulangan dari submasalah terkecil.
D. Bottom-up selalu lebih lambat secara asimtotik.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Divide and Conquer juga memecah masalah (A) dan juga rekursif (D salah). Pembedanya adalah tumpang tindih: Dynamic Programming menyimpan solusi submasalah yang muncul berulang.

Soal 2

Optimal substructure berarti solusi optimal suatu masalah memuat solusi optimal dari submasalahnya. Sifat ini diperlukan karena Dynamic Programming menyusun solusi besar dari nilai-nilai optimal submasalah yang disimpan di tabel. Jika solusi optimal masalah besar ternyata membutuhkan solusi submasalah yang **tidak** optimal, maka menggabungkan nilai-nilai optimal di tabel tidak akan menghasilkan jawaban yang benar.

Soal 3

Banyaknya F_2 dihitung: $F_6 \to F_5, F_4$; secara umum banyaknya pemanggilan F_k dalam menghitung F_n adalah F_{n-k+1}. Untuk $k = 2$, $n = 6$: $F_5 =$ **5 kali**. Total pemanggilan $= 2F_7 - 1 = 2 \cdot 13 - 1 =$ **25**.

Soal 4

$r_5 = \max(p_1 + r_4,\ p_2 + r_3,\ p_3 + r_2,\ p_4 + r_1,\ p_5 + r_0) = \max(1 + 10,\ 5 + 8,\ 8 + 5,\ 9 + 1,\ 10 + 0) = \max(11, 13, 13, 10, 10) =$ **13**. Caranya: potongan 2 dan 3 ($5 + 8 = 13$).

Soal 5

Salah. Langkah 4 hanya dibutuhkan bila kita ingin mengetahui solusinya, bukan hanya nilainya. Jika yang ditanyakan hanya "berapa operasi minimal" atau "berapa panjang LCS", langkah 1–3 sudah cukup.

Soal 6

Ada n^2 sel, masing-masing butuh $O(n)$ pemeriksaan, sehingga running time-nya $O(n^3)$. Memorinya sebanding dengan ukuran tabel, yaitu $\Theta(n^2)$. (Ini persis situasi MATRIX-CHAIN-ORDER di Modul 11.)

Soal 7

Jawaban: C. Keduanya menyimpan hasil submasalah (A salah). Justru memoization yang hanya menghitung submasalah yang dibutuhkan (B salah). Running time asimtotik keduanya umumnya sama (D salah).

Referensi

Sumber utama: Materi 8 DYNAMIC PROGRAMMING Perkalian Rantai Matriks (bagian pengantar Dynamic Programming) (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Bellman, R. (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.
3. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Boston: Pearson/Addison-Wesley.